2025 IEEE International Students' Conference on Electrical, Electronics and Computer Science (SCEECS) | 979-8-3315-2983-3/25/$31.00 ©2025 IEEE | DOI: 10.1109/SCEECS64059.2025.10940886

Stress Detection Using SVM and Virtual Assistant

Pranav Amale
Electronics and Telecommunications Department
Dr. D. Y. Patil Institute of Technology
Pimpri, Pune
panuamale.2754@gmail.com

Anup Vibhute
Instrumentation Engineering Department
Dr. D. Y. Patil Institute of Technology
Pimpri, Pune
anup.vibhute@dypvp.edu.in

Mahesh S. Mathpati
Electronics and Telecommunications Department
SVERIs College of Engineering, Pandharpur
msmathpati@coe.sveri.ac.in

Ashwini V Jatti
Instrumentation Engineering Department
Dr. D. Y. Patil Institute of Technology
Pimpri, Pune
Ashwini.jatti@dypvp.edu.in

Somnath B. Thigale
Computer Science and Engineering Department
Fabtech College of Engineering, Sangola
drsomnaththigale@gmail.com

***Abstract*— This paper presents a chatbot model designed to detect stress in users through simple yes-or-no questions. Utilizing a Support Vector Machine (SVM) algorithm, the model achieves high accuracy in identifying stress levels and provides users with actionable advice or medical recommendations. The chatbot's user-friendly interface is built using Anaconda Navigator and Python, ensuring ease of use for non-technical users. The model achieves a 100% accuracy rate on the given dataset, making it a promising tool for stress detection and early intervention.**

Keywords—SVM, stress detection, virtual assistant, mental health, predictive model, accuracy, Python, early intervention.

I. Introduction

Stress is a common issue that affects millions of people worldwide. It can be caused by work pressures, personal problems, or societal challenges and, over time, can lead to serious health issues like anxiety, depression, and heart disease. Although stress is widespread, many people do not recognize when they are stressed, and others may not seek professional help due to stigma, cost, or lack of access to healthcare. This makes it crucial to find new, accessible ways for individuals to assess their stress levels early, before they lead to more serious health conditions.

Currently, stress detection is mostly done through surveys or interviews, where people are asked to describe their feelings and experiences. However, these methods are not always accurate because they rely on how well individuals can recognize their own stress and how honestly they answer the questions.[1] Many people may not fully understand or be aware of their stress, while others may find it difficult to openly discuss their emotional state. As a result, traditional methods often fail to detect stress early, which delays necessary intervention and treatment[1-3].

In recent years, there has been a shift towards using technology to help people manage their mental health, particularly through chatbots. Chatbots are computer programs that can simulate conversations with users. They are becoming increasingly popular because they are easy to use, available 24/7, and provide a private and accessible way for people to talk about their feelings without fear of judgment[1]. A chatbot can help individuals identify their stress levels through simple yes/no questions, making it easier to monitor their mental health on a regular basis.

One promising way to improve stress detection in chatbots is by using machine learning algorithms. These algorithms can analyze the user's responses and determine whether the person is likely experiencing stress. Specifically, this study focuses on using a machine learning model called Support Vector Machine (SVM) to help the chatbot recognize patterns in users' answers and predict whether they are stressed[2]. The goal is to create a chatbot that can not only ask simple questions but also provide useful feedback and suggestions for stress management based on the answers it receives.[4]

II. System Specification and Block Diagram

2.1 External Interface Requirements

2.1.1 User Interface

The system will use an application-based chatbot for easy user interaction. It will ask basic questions to assess the user's stress levels[1].

2.1.2 Hardware Interfaces

The system requires more than 8 GB RAM, free HDD space over 40 GB for storing CT Scan data, and an Intel Core i5 processor for quick data processing. The development environment will be PyCharm, and the programming language used will be Python 3.5.

2.1.3 Software Interfaces

The system will run on Windows 10 with PyCharm and Spyder as the development environments, and Python for coding.

2.2 Non-Functional Requirements

2.2.1 Performance Requirements

The system must respond within 2 seconds to user input and handle at least 1000 transactions per minute.

2.2.2 Safety Requirements

The system must ensure privacy and offer correct advice to avoid increasing stress.

979-8-3315-2983-3/25/$31.00 ©2025 IEEE

Authorized licensed use limited to: REVA UNIVERSITY. Downloaded on December 18,2025 at 05:05:23 UTC from IEEE Xplore. Restrictions apply.

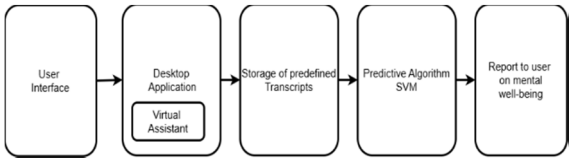

Fig 2.1. Block Diagram for Mental Health Assessment System

As shown in Figure 2.1, the user interface is where users use the system. It uses a simple conversational format to ask yes-or-no questions about the user's mental health and well-being. The desktop application with virtual assistant provides a platform for the user to respond to these questions. It incorporates a conversational interface, facilitating user engagement with the system.

The repository of preconfigured transcripts is a database that contains potential replies and associated labels for mental health. This assists the system in analyzing and referencing responses throughout the review process. The statistical models SVM-Support Vector Machine, which evaluates the responses [4]. The system utilizes the archived transcripts to forecast the user's mental health condition based on their responses.

Ultimately, the system delivers input to the user regarding their mental health status. The analysis generates a personalized report regarding the user's mental health component. It provides recommendations, guidance and resources for early intervention and support, assisting the user in comprehending their mental health condition.

III. Methodology

The procedure begin when the user engages with the virtual assistant via a conversational interface. The assistant poses binary questions aimed at comprehending the user's psychological condition. The responses are archived alongside predetermined transcripts that offer supplementary information.

The acquired data is subsequently input into a machine learning technique such as support Vector Machine. The SVM examines the data to construct a predictive model that assesses the user's mental well-being by recognizing patterns in their replies.[5]

After the SVM analysis, the virtual assistant produces a comprehensive report of the user. This report provides insights, feedback, and resources customized to the user's mental health need. It assists in recognizing possible issues and proposes methods for early intervention and assistance.

The objective of the project is to enhance mental well-being by providing a customized, non-invasive instrument for self-evaluation.[3] The SVM algorithm enables the virtual assistant to assess mental health and produce reports to aid users in maintaining their well-being. This approach emphasizes the capability of virtual assistants and machine learning to identify mental health concerns early and provide valuable tools for sustaining optimal mental health.

IV. Software Design

The system is organized into multiple modules to efficiently process and address customer inquiries.

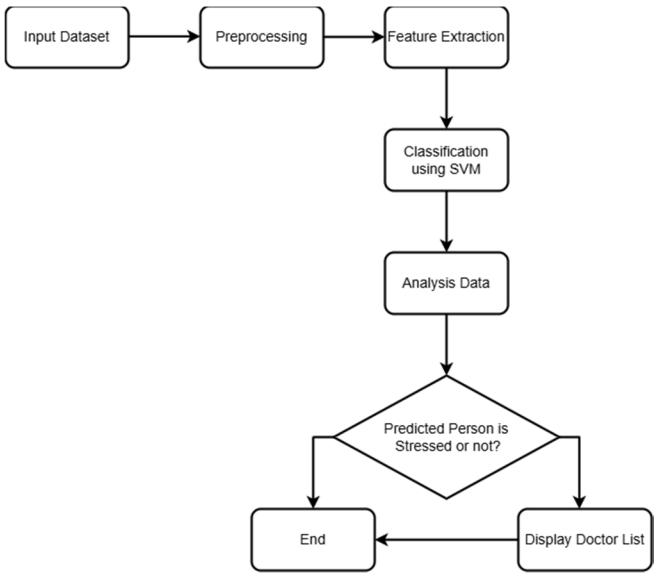

Fig 4.1. System Architecture

In Figure 4.1, the user submits a question or query to the system. The system subsequently identifies the query text and processes it via text processing methodologies. The acknowledged material is thereafter transformed into a functional format and translated if required. The system subsequently aligns the query with stored data utilizing the Support Vector Machine method, and ultimately, a response is produced and promptly converted into the suitable format for the user. The Software Development Life Cycle (SDLC) directs this process by delineating the necessary processes for software development. The SDLC encompasses modules like requirements collection and analysis, when the development team collaborates with stakeholders to delineate functionalities. The system design module subsequently converts requirements into comprehensive architectural and user interface specifications. The development step entails coding according to these concepts, succeeded by thorough testing to verify the software operates as planned. Upon testing, the software is deployed to the production environment, and further operations, including bug repairs and upgrades, are managed throughout the maintenance and support phase. Furthermore, the system employs Support Vector Machine (SVM), a robust technique for categorization [4].With a dataset of m samples $(x_1, y_2),(x_2,y_2),\ldots,(x_m, y_m)$, where x_i is the input feature and y_i is the class label, SVM looks for a hyperplane that has the most space between the two classes.[5] This is solved as an optimization problem-

$$\text{minimize } \frac{1}{2}\|\omega\|^2 + c\sum_{i=1}^{m} max\big(0,1 - y_i(\omega^T x_i + b)\big)$$

In this case, w the weight vector, b the bias, and C the regularization measure that makes the margin size and designation error equal. The decision function is defined as:

$$f(x) = sign(\omega^T x + b)$$

where sign() returns +1 or -1 depending on the value of $\omega^T x + b$, determining the class of the input sample.

Authorized licensed use limited to: REVA UNIVERSITY. Downloaded on December 18,2025 at 05:05:23 UTC from IEEE Xplore. Restrictions apply.

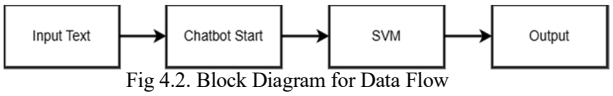

Fig 4.2. Block Diagram for Data Flow

Figure 4.2 is a picture of how data moves through a system or process. From where the data comes from to where it ends up, it keeps track of its flow through different systems.[6] In system analysis and design, DFDs are often used to show how information moves through a system. They give a clear, high-level picture of how the system works and how its different parts interact with each other[5]. When developers use a DFD, they can better see how data is processed and find places in the system where problems or changes could be made.

V. Test Cases

Test cases are meant to check that key parts of a software system work properly and are safe. The purpose of GUI test cases is to test the application's graphical user interface (GUI).

Test Case	Login Screen-Sign up
Objective	Click on sign up button then check all required mandatory fields with leaving all fields blank
Expected Result	All required mandatory fields should display with symbol Instruction line "* field(s) are mandatory" should be displayed
Test case	Create a Password >>Text Box Confirm Password >>Text Box
Objective	Check the validation message for Password and Confirm Password field
Expected Result	Correct validation message should be displayed accordingly or "Password and confirm password should be same" in place of "Password mismatch.

Fig 5.1. GUI Test Cases

Figure 5.1 illustrates test cases aim to confirm that the GUI is user-friendly, functional, and meets the design requirements. Usually, each test case has a name, a short description, the steps that need to be taken to run it, the results that are expected, and clear criteria for figuring out whether the test passed or failed. These tests ensure that the user interface operates as expected, providing a seamless experience for users.

Test Case ID	Test Case	Test Case I/P	Actual Result	Expected Result	Test case criteria(P/F)
001	Enter the wrong username or password click on submit button	Username or Password	Error comes	Error Should come	P
002	Enter the correct username and password click on submit button	Username and Password	Accept	Accept	P

Fig 5.2. GUI Test Cases

To make sure the login process works right, you need login test cases. These test cases are shown in Figure 5.2. They check important parts of the page like the username and password fields, the login button, and security features like CAPTCHA, two-factor authentication, and the "forgot password" link. For these tests to work, a user must have finished the registration process in order to be able to log in. In order to register, the database is told what information it needs, like the account and password. Because of this, users can only log in after giving all the necessary information when they registered. The login test cases also check how many failed tries there can be to log in, making sure that the system keeps people from getting in without permission and stays safe[4].

Test Case	Test Case	Test Case I/P	Actual Result	Expected Result	Test Criteria(P/F)
001	Enter the number in username, middle name, last name field	Number	Error Comes	Error Should Comes	P
001	Enter the number in username, middle name, last name field	Character	Accept	Accept	P
002	Enter the invalid email id format in email id field	kkgmail.com	Error Comes	Error Should Comes	P
002	Enter the valid email id format in email id field	kk@gmail.com	Accept	Accept	P
003	Enter the invalid digit number in phone number field	99999	Error Comes	Error Should Comes	P
003	Enter the 10 digit number in phone number field	9999999999	Accept	Accept	P

Fig 5.3. Test Cases

There are registration test cases in Figure 5.3 that check how well the process of registering a person works. They include a number of important situations to make sure that new users can easily sign up. Valid registration test cases, for instance, make sure that users can successfully join by giving all the necessary information, like an email address, a username, and a password. If a person enters all the required information, the system should either let them move on or let them know that the registration is complete. Checking that the system gives the right error messages if a user tries to register without filling in required fields is one way to test empty fields. Password complexity tests make sure that the system follows strong password rules by rejecting weak passwords and showing mistake messages when they're needed. Email validation test cases make sure that the system can spot and dismiss emails that don't follow the rules, like ones with wrong domain names or missing symbols. Lastly, username uniqueness test cases make sure that the system doesn't create two identical usernames. If a person tries to register with a username that's already been taken, they will see an error message. These test cases help make sure that the registration process is safe and works, which stops problems like usernames or passwords that don't work together.

Authorized licensed use limited to: REVA UNIVERSITY. Downloaded on December 18,2025 at 05:05:23 UTC from IEEE Xplore. Restrictions apply.

VI. Result and analysis

Using a chatbot, this model is meant to find people who are stressed. The SVM method is used to train the model, and it is built in Python. An experimental setup with 8GB of RAM, 40GB of hard disk space, database software, Python, and Anaconda Navigator is used. The model checks to see if a person is stressed by asking them 10 easy yes/no questions.[2] If stress is found, the model offers home remedies and links to doctors for more help. The model did amazingly well, with an F1 score of 1.00 showing that it correctly predicted stress in all test cases[3].

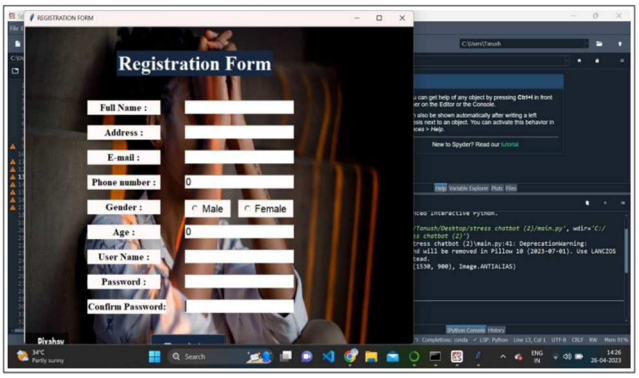

Fig 6.1. Registration Diagram

The model's real-time operation begins on the primary graphical user interface (GUI) page, where users may either register or log in. Figure 6.1 illustrates the registration screen, where users input their credentials to enroll in the testing process. Upon registration, users may log in using the GUI and advance to the testing page.

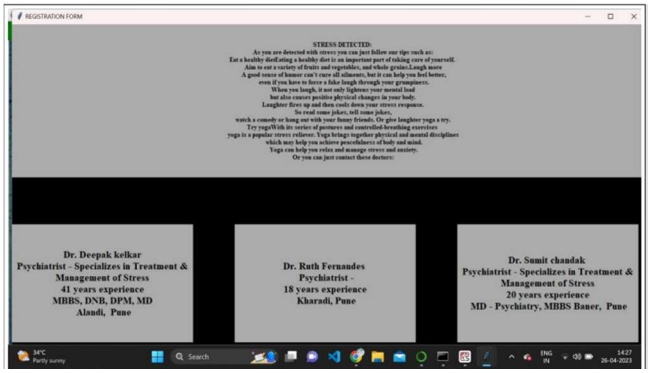

Fig 6.2. Output Diagram

Upon logging in, users are directed to the master page, where they can either initiate the test or exit. The chatbot interface poses a sequence of yes/no inquiries, and following the submission of responses, the model evaluates the answers to ascertain the presence of stress.[3] Following stress detection, Figure 6.2 illustrates the interface presenting solutions and physician recommendations for consumers to alleviate their stress.

	precision	recall	f1-score	support
0	1.00	1.00	1.00	1217
1	1.00	1.00	1.00	2185
2	1.00	1.00	1.00	1487
accuracy			1.00	4889
macro avg	1.00	1.00	1.00	4889
weighted avg	1.00	1.00	1.00	4889

Accracy : 100.0%
Model saved as SVM.joblib

Fig 6.3. Accuracy Diagram

The model's efficacy is additionally assessed by metrics like precision, recall, and F1 score. Figure 6.3 illustrates the efficacy of the SVM-based model, attaining an impeccable score of 1.00 for both precision and recall. Precision denotes that all affirmative predictions were accurate, whereas recall demonstrates that the model accurately recognized all occurrences of stress. The F1 score balances precision and recall, also stands at 1.00, reflecting excellent performance. The macro and weighted averages of precision, recall, and F1 score are all 1.00, indicating that the model performs consistently well across all classes in the dataset. The support value, representing the number of samples, is 4889, confirming the model's reliability on a large dataset.[2]

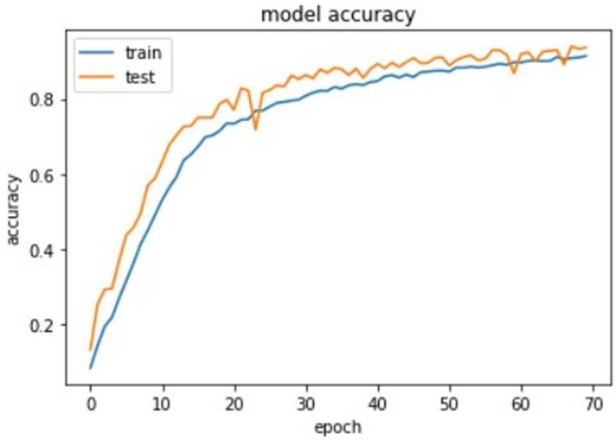

Fig 6.4 Accuracy Graph

Accuracy tells us the percentage of correct predictions made by the model. In this case, the model achieved 100% accuracy, meaning all the test instances were classified correctly. However, while accuracy is useful, it may not fully reflect the performance, particularly in scenarios with imbalanced datasets or when Different errors, such as false positives and false negatives have varying consequences. In such cases, precision, recall, and F1 score are also valuable metrics. Factors affecting accuracy include data quality, the choice of algorithm, model hyperparameters, and the features used for training.

The model demonstrates exceptional performance in detecting stress with a high level of accuracy and precision.

Authorized licensed use limited to: REVA UNIVERSITY. Downloaded on December 18,2025 at 05:05:23 UTC from IEEE Xplore. Restrictions apply.

This analysis highlights the effectiveness of using techniques like SVM for real-time applications such as stress detection, offering both practical utility and clinical relevance.

VII. CONCLUSION

This paper presents a virtual assistant-driven predictive model for mental well-being using the SVM algorithm. The model asks yes/no questions to assess a user's mental state and accurately predicts whether they are stressed, offering personalized feedback, doctor recommendations, and home remedies. With a 100% accuracy rate and an F1 score of 1.00, the system is highly effective. The framework is designed to be user-friendly and capable of handling millions of clients simultaneously, making it a notable tool for early detection and prevention of mental health issues.

VIII. REFERENCES

[1] J. L. L. a. S. R. Tamizharasi B., "Building a Medical Chatbot using Support Vector Machine Learning Algorithm," in *National Science, Engineering and Technology Conference (NCSET)*, Chennai, India, 2020.

[2] P. L. S. P. T. S. a. K. R. P. Namrata Khatape, "Stress Detection System using the SVM Algorithm," *Journal of Advances in Computational Intelligence Theory,* vol. 3, no. 1, pp. 1-5, 2021.

[3] S. G. a. N. P. T. Anu Priya, "Predicting Anxiety, Depression and Stress in Modern Life using Machine Learning Algorithms," in *International Conference on Computational Intelligence and Data Science (ICCIDS)*, Gurgaon, India, 2019.

[4] R. F. a. M. H. I. Md Fahim Rizwan, "Support Vector Machine based Stress Detection System to manage COVID-19 pandemic related stress from ECG signal," *AIUB JOURNAL OF SCIENCE AND ENGINEERING,* Vols. AJSE COVID-19 Special Issue, pp. 8-16, 2020.

[5] S. a. P. S. R. Jegan, "Mental Stress Detection and Classification using SVM Classifier : A Pilot Study," in *6th International Conference on Devices, Circuits and Systems (ICDCS)*, Coimbatore, India, 2022.

[6] C.-C. C. a. Y.-T. T. Chien-Hao Kao, "Model of Multi-turn Dialogue in Emotional Chatbot," in *International Conference on Technologies and Applications of Artificial Intelligence (TAAI)*, Kaohsiung, Taiwan, 2019.

Authorized licensed use limited to: REVA UNIVERSITY. Downloaded on December 18,2025 at 05:05:23 UTC from IEEE Xplore. Restrictions apply.